28. Database Schema Evolution

Key Terms:

Term	Full Name	What it does
Migration	Schema change script	Versioned script that modifies database structure
Expand-Contract	Migration pattern	Add new → migrate data → switch → remove old
Backward Compatible	Safe change	Old app versions still work after change
gh-ost	GitHub Online Schema Transformation	Tool for zero-downtime MySQL migrations
Binlog	Binary log	MySQL log of all changes (used for replication)

Problem Statement

Design strategies for evolving database schemas over time without downtime or data loss.

Clarifying Questions to Ask

1. **Downtime**: Can you have maintenance windows?
2. **Scale**: How much data? How many tables?
3. **Frequency**: How often do schemas change?
4. **Rollback**: Need to roll back migrations?
5. **Testing**: How to test migrations safely?
6. **Multi-version**: Multiple app versions running?
7. **Replication**: Cross-region replication involved?

API Design

Migration Management APIs

Method	Endpoint	Description

`POST`	`/api/v1/migrations`	Create and queue a migration
`GET`	`/api/v1/migrations`	List all migrations
`GET`	`/api/v1/migrations/{migrationId}/status`	Get migration status
`POST`	`/api/v1/migrations/{migrationId}/execute`	Execute a pending migration
`POST`	`/api/v1/migrations/{migrationId}/rollback`	Rollback a migration

Schema APIs

Method	Endpoint	Description
`GET`	`/api/v1/schemas/{table}/current`	Get current schema
`GET`	`/api/v1/schemas/{table}/history`	Get schema change history
`POST`	`/api/v1/schemas/{table}/validate`	Validate a proposed schema change

API Versioning for Schema Changes

```
// Support both old and new schema during migration
GET /api/v2/users    → { "full_name": "John Doe" }
GET /api/v1/users    → { "first_name": "John", "last_name": "Doe" }
```

Migration Strategies

```
1. EXPAND-CONTRACT PATTERN (Zero downtime)
┌─────────────────────────────────────────────────────────────────┐
│  Rename column: username → user_name                            │
│                                                                  │
│  PHASE 1: EXPAND (add new column)                               │
│  ┌──────────────────────────────────────────────────────┐       │
│  │  ALTER TABLE users ADD COLUMN user_name VARCHAR(255);  │     │
│  │  -- App writes to BOTH columns                        │     │
│  │  -- App reads from username (old)                     │     │
│  └──────────────────────────────────────────────────────┘       │
│                                                                  │
│  PHASE 2: MIGRATE (backfill data)                               │
│  ┌──────────────────────────────────────────────────────┐       │
│  │  UPDATE users SET user_name = username                │     │
│  │  WHERE user_name IS NULL;                             │     │
│  │  -- Batch to avoid locking                            │     │
│  └──────────────────────────────────────────────────────┘       │
│                                                                  │
```

```
│  PHASE 3: TRANSITION (switch reads)                          │
│  ┌──────────────────────────────────────────────────────┐    │
│  │  -- Deploy new app version                           │    │
│  │  -- App reads from user_name (new)                   │    │
│  │  -- App still writes to BOTH (for rollback)          │    │
│  └──────────────────────────────────────────────────────┘    │
│                                                              │
│  PHASE 4: CONTRACT (remove old column)                       │
│  ┌──────────────────────────────────────────────────────┐    │
│  │  ALTER TABLE users DROP COLUMN username;             │    │
│  │  -- Only after all app versions updated              │    │
│  └──────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────┘
```

2. ADDITIVE CHANGES ONLY

```
┌──────────────────────────────────────────────────────────────┐
│  Safe (backward compatible):                                 │
│    ✓ Add nullable column                                     │
│    ✓ Add table                                               │
│    ✓ Add index (CREATE INDEX CONCURRENTLY)                   │
│    ✓ Widen column (VARCHAR(50) → VARCHAR(100))               │
│                                                              │
│  Unsafe (requires expand-contract):                          │
│    ✗ Remove column                                           │
│    ✗ Rename column                                           │
│    ✗ Change column type                                      │
│    ✗ Add NOT NULL constraint                                 │
└──────────────────────────────────────────────────────────────┘
```

Migration Tools

MIGRATION FILE STRUCTURE:

```
┌──────────────────────────────────────────────────────────────┐
│  migrations/                                                 │
│  ├── 001_create_users_table.sql                              │
│  ├── 002_add_email_to_users.sql                              │
│  ├── 003_create_orders_table.sql                             │
│  └── 004_add_user_name_column.sql                            │
│                                                              │
│  Each migration:                                             │
│    - Has unique version number                               │
│    - Runs exactly once                                       │
│    - Tracked in migrations table                             │
└──────────────────────────────────────────────────────────────┘
```

MIGRATION TRACKING TABLE:

```
┌──────────────────────────────────────────────────────────────┐
│  CREATE TABLE schema_migrations (                            │
│    version VARCHAR(255) PRIMARY KEY,                         │
│    applied_at TIMESTAMP DEFAULT NOW()                        │
│  );                                                          │
│                                                              │
│  -- Check what's applied                                     │
```

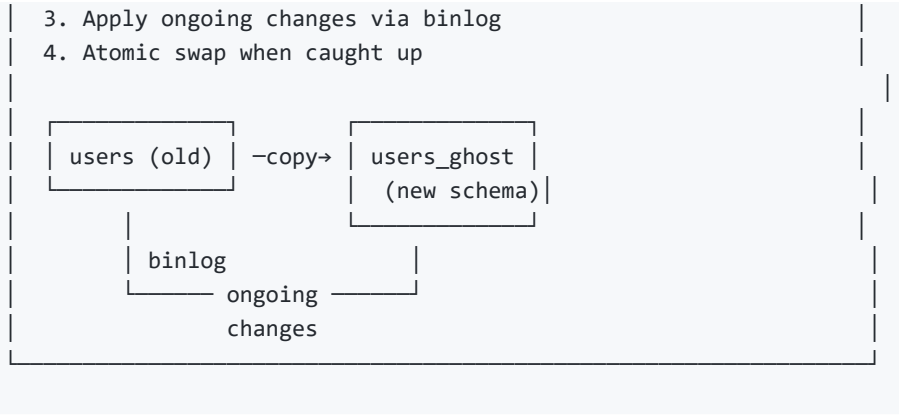
```
┌──────────────────────────────────────────────────────────────────┐
│  SELECT * FROM schema_migrations;                                │
│  | version | applied_at          |                              │
│  |---------|---------------------|                              │
│  | 001     | 2024-01-01 10:00:00 |                              │
│  | 002     | 2024-01-05 14:30:00 |                              │
│  | 003     | 2024-01-10 09:15:00 |                              │
└──────────────────────────────────────────────────────────────────┘
```

Large Data Migrations

```
BATCHED MIGRATION (Avoid locking):
┌──────────────────────────────────────────────────────────────────┐
│  -- DON'T: Update all at once (locks table)                      │
│  UPDATE users SET new_col = old_col;                             │
│                                                                  │
│  -- DO: Batch with limits                                        │
│  DO $$                                                           │
│  DECLARE                                                         │
│    batch_size INT := 1000;                                       │
│    affected INT;                                                 │
│  BEGIN                                                           │
│    LOOP                                                          │
│      UPDATE users                                                │
│      SET new_col = old_col                                       │
│      WHERE new_col IS NULL                                       │
│      AND id IN (                                                 │
│        SELECT id FROM users                                      │
│        WHERE new_col IS NULL                                     │
│        LIMIT batch_size                                          │
│        FOR UPDATE SKIP LOCKED                                    │
│      );                                                          │
│                                                                  │
│      GET DIAGNOSTICS affected = ROW_COUNT;                       │
│      EXIT WHEN affected = 0;                                     │
│                                                                  │
│      PERFORM pg_sleep(0.1);  -- Breathing room                   │
│    END LOOP;                                                     │
│  END $$;                                                         │
└──────────────────────────────────────────────────────────────────┘
```

Online Schema Changes (Large Tables)

```
TOOLS FOR ZERO-DOWNTIME:
┌──────────────────────────────────────────────────────────────────┐
│  MySQL: pt-online-schema-change, gh-ost                          │
│  PostgreSQL: pg_repack, pgroll                                   │
│                                                                  │
│  How gh-ost works:                                               │
│  1. Create ghost table with new schema                           │
│  2. Copy data in chunks                                          │
```



Trade-offs

Decision	Option A	Option B	Recommendation
Downtime	Maintenance window	Zero-downtime	**Zero** for production
Approach	Direct ALTER	Expand-contract	**Expand-contract** - safer
Data migration	Synchronous	Background job	**Background** for large tables
Rollback	Support	No support	**Support** - always have undo

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
PostgreSQL	Primary database for schema migrations	Tech Guide

How to Explain It Well

> **Opening**: "Schema evolution must be backward-compatible during deployment. I use expand-contract for breaking changes."

> **Key talking points**:
>
> - "Expand-contract: add new, migrate, switch, remove old"

> - "Only additive changes are safe: new columns, tables, indexes"
> - "Batch large data migrations to avoid table locks"
> - "Tools like gh-ost for online schema changes on large tables"
> - "Track migrations in database, never rely on manual execution"
